Baseball Trade Recommendation Platform

Venkata Krishna Kandalai

August 15, 2026

CONTENTS

Baseball Trade Recommendation Platform

A Complete Guide: What It Does, Why It's Built This Way, and How to Defend It

---

Part 1: The Big Picture

What is this, in one sentence?

A pipeline that looks at every Major League Baseball player, predicts how good they'll be, and calculates a single dollar number representing how good a trade target they are — then uses that number to find realistic trades between teams.

The core idea, before any technical detail

Imagine two players who are both excellent. One of them is a 32-year-old superstar making $35 million a year. The other is a 23-year-old who is nearly as good, but still on a cheap rookie contract making $800,000 a year.

If you were a general manager, which one would you rather **trade for**?

Most people's instinct is "the superstar, obviously — he's better." But that's the wrong question. The right question is: **who gives you the most value relative to what you have to pay them?** The 23-year-old does. The superstar's price tag already reflects how good he is — there's no bargain there. The rookie is a bargain, because his paycheck hasn't caught up to his talent yet.

This project exists to compute exactly that number — called **surplus value** — for every player in baseball, so trade decisions can be grounded in "who's a bargain," not just "who's good."

The five-stage pipeline

```
INGEST  →  RESOLVE  →  FORECAST  →  VALUATION  →  RECOMMEND
(get       (match       (predict     (turn a       (find
 the        names to     next         prediction     realistic
 data)      real         season's     into a         trades)
            players)     value)       dollar
                                      figure)
```

Each stage is a separate program. Each one reads from and writes to a shared database, so the whole thing can be re-run end to end, or any single stage can be re-run on its own once its inputs change.

Part 2: Baseball, Explained Simply

You don't need to know baseball to follow the rest of this document, but a few terms come up constantly.

Batters and pitchers are the two broad categories of player, and they're evaluated on almost entirely different sets of statistics (how often a batter gets a hit vs. how few runs a pitcher allows). This project trains two separate prediction models — one per category — because mixing them wouldn't make sense.

WAR (Wins Above Replacement) is baseball's attempt at a single number that answers "how many extra wins did this player's presence add to their team's season, compared to a freely available replacement-level player?" It bundles together hitting, pitching, baserunning, and fielding into one number. A great player might be worth 6+ WAR in a season; an average player is often worth 1-2; a player who's actively hurting their team can have negative WAR.

"Replacement level" is the baseline WAR is measured against — not an average player, but the caliber of a player any team could call up from the minor leagues for free at a moment's notice. This matters: if you measured against the *average* player instead, half the league would show up as "below average" even though they're still valuable — replacement level is a much lower, more honest bar.

Fielding/defense is part of a player's value that's easy to forget about if you're only looking at batting stats (hits, home runs, etc.). A player who's excellent defensively adds real value even with an unremarkable bat. This turned out to be a real blind spot in an early version of the model — more on that in Part 4.

Surplus value (this project's core output) = the dollar value of a player's *projected future performance*, minus what they're actually being *paid*. Positive and large = a great trade target. Positive and small, or negative = not a bargain, even if the player themselves is good.

Part 3: Machine Learning, Explained Simply

A "model" here is just a mathematical function that's been tuned on historical examples to guess an unknown output (a player's WAR) from known inputs (their stats). Think of it as a very sophisticated rule of the form "players with stats like *this* tend to have WAR like *that*," learned automatically rather than hand-written.

Training vs. testing — a model is *trained* on data where the right answer is already known (past seasons), so it can find the pattern. It's then *tested* on different data it never saw during training, to check whether it actually learned a real pattern or just memorized the training examples. Testing on the same data you trained on is like grading a student using the exact questions they already saw the answer key for — it always looks better than it really is.

MAE (Mean Absolute Error) — on average, how far off was the model's guess from the real answer? If MAE = 0.5 WAR, the model is typically off by about half a win. Smaller is better.

R^2 ("R-squared") — of all the real, natural differences between players (some are great, some are average, some are bad), what percentage of that spread does the model actually explain? 100% = perfect. 0% = the model isn't finding any real pattern at all, no better than a random guess. This is the headline number used throughout this project to describe model quality.

A "naive baseline" is a deliberately dumb comparison point — e.g., "just guess the league-average player" or "just guess the player repeats what they did last year." If a real model can't beat a naive baseline, it isn't actually adding value, no matter how sophisticated it looks.

Part 4: The Pipeline, Stage by Stage — Every Design Decision and Why

This is the core of interview prep: not just *what* was built, but *why* each choice was made, and what the alternative would have cost.

Stage 1: Ingest

What it does: pulls the player ID registry, 12 seasons (2014-2025) of real historical stats and actual WAR, and the incoming season's projections, from public baseball data sources, into a local database.

DESIGN DECISION: EVERY FACT IS TIMESTAMPED ("POINT-IN-TIME" DATA)

Every single row of data carries an `as_of_date` — the date it was known to be true. This is the single most important design decision in the whole project, and it's worth being able to explain cold:

> A model that's tested using information from *after* the decision point would look artificially good — it's cheating by seeing the future. This is called **look-ahead bias**, and it's one of the most common, most dangerous mistakes in any predictive system built on historical data (not just baseball — the same failure mode ruins stock-trading backtests and medical risk models). Stamping every fact with when it was known makes it possible to ask "what did we actually know at the time?" and get a truthful answer, rather than an accidentally-cheating one.

DESIGN DECISION: RETRY AND PACING ON EXTERNAL DATA PULLS

The historical stats come from Baseball-Reference via a public library. Pulling 12 years of data with no delay between requests gets you rate-limited (blocked) partway through — and a naive version of this code failed *silently*, logging a warning and moving on, leaving multiple years of training data quietly missing. The fix: space requests out, retry with increasing wait times on failure, and cache successful pulls so a rerun doesn't waste time re-fetching data it already has. **The lesson worth stating out loud in an interview:** a pipeline that fails quietly is worse than one that fails loudly — the silent version can run for weeks producing subtly wrong results before anyone notices.

DESIGN DECISION: SNAPSHOT TO DISK BEFORE WRITING TO THE DATABASE

Every ingest run also saves a timestamped copy of the raw data to disk (`data/snapshots/`). This means the exact data behind any past run of the model can be reconstructed later, independent of whatever's currently in the live database — useful for debugging, and for the point-in-time guarantee above to actually mean something.

Stage 2: Resolve

The problem: the incoming projections list players by name — "Bobby Witt Jr.," "Matt Boyd" — as plain text. To connect that to 12 years of historical performance, each name has to be matched to a permanent player ID. This sounds trivial and isn't: nicknames ("Matthew Boyd" vs. the official "Matt Boyd"), and two entirely dif-

ferent real players sharing a name (there are two MLB players named “Luis Castillo”) both cause silent wrong matches if handled naively.

DESIGN DECISION: A LAYERED MATCHING WATERFALL, MOST CONFIDENT FIRST

1. **Exact match** — the name matches exactly one player in the registry. Done.
2. **Narrow by recent activity** — multiple name matches, but only one of them has played in the last 3 years. Use that one.
3. **Narrow by team** — still multiple active candidates, but only one has recently played for the team the projection lists. Use that one.
4. **Fuzzy text match** — no exact match, but a very close text match against currently active players clears a confidence threshold.
5. **Give up rather than guess** — if none of the above resolves it confidently, the system marks it unresolved rather than picking a low-confidence guess.

Why this matters, and the interview-ready line: *”Silent join failures produce wrong surplus value calculations that look completely normal — there’s no error, just a quietly wrong number. An 85% automatic match rate that’s honest about its 15% gap is more trustworthy than a 100% match rate that’s silently guessing wrong on some of them.”*

DESIGN DECISION: A MANUAL OVERRIDES FILE FOR THE CASES AUTOMATION SHOULDN’T GUESS

Running this on the current season’s data left 14 unresolved names. Digging into them by hand revealed two very different situations:

- **10 were genuine, unavoidable gaps** — just-drafted prospects with no MLB ID assigned yet, since they haven’t debuted. Nothing to fix; they resolve automatically the moment they do debut.
- **4 were real players the algorithm got wrong for fixable reasons** — two nickname mismatches (the registry uses “Matt”/”Mike,” the projection used “Matthew”/ “Michael”), and two genuine name collisions between different real players, resolved by hand-checking which one actually played for the listed team.

A small CSV file (`data/manual_overrides.csv`) captures both categories — confirmed IDs for the 4 fixable ones, and an honest “not yet possible” flag for the 10 prospects — and the resolver checks it first, before falling back to automated matching. **Result: 793 of 803 players (98.8%) resolved correctly.**

Stage 3: Forecast — the WAR Prediction Model

What it does: trains a model on 12 years of real stat-line → real-WAR pairs, then uses that trained model to predict next season’s WAR from each player’s incoming projected stat line.

DESIGN DECISION: TWO SEPARATE MODELS (BATTERS AND PITCHERS)

Batters and pitchers are scored on entirely different stats, so one model tries to learn "PA, hits, home runs, walks → WAR," and a separate model learns "innings pitched, ERA, strikeout rate → WAR." Forcing one model to handle both would blur two genuinely different relationships together.

DESIGN DECISION (AND LESSON): THE FIELDING BLIND SPOT

An early version of the batting model only used offensive stats (hits, walks, home runs, etc.) as inputs — even though the actual target it was trying to predict, WAR, already includes fielding value. That's a real gap: a plus defensive shortstop and a poor defensive shortstop with identical batting lines would get the exact same prediction, because the model literally couldn't see the difference.

The fix required getting one subtlety right: **you cannot use a player's fielding value from the *same* season you're predicting.** That season's fielding is literally part of what makes up that season's WAR — using it as an input would be handing the model part of the answer (a mistake called **data leakage**). The correct fix: use each player's fielding performance from their **most recent previous season** as a feature. That's not just safer, it's also the only thing actually available at real prediction time, since incoming projections don't forecast fielding at all.

Proof it actually helped, not just moved rankings around: removing the feature and re-testing dropped batting accuracy from R^2=0.375 down to R^2=0.331 — a real, measured accuracy loss, not just a coincidental reshuffling of who's ranked where.

DESIGN DECISION: ADDITIONAL BATTING INPUTS FOUND BY AUDITING THE RAW DATA

A later audit found that the incoming projection spreadsheet actually contained more columns than the code was using — runs scored, RBIs, doubles, triples, and caught-stealing were present in the raw file but silently dropped during ingestion, even though the matching historical columns already existed in the training data. Adding them (age too, which was already available but unused) measurably improved batting accuracy (R^2=0.358 → 0.375). **The general lesson:** before reaching for a fancier model, audit whether you're actually using all the signal already sitting in your own data.

DESIGN DECISION: DIFFERENT MODEL TYPES PER POSITION GROUP

Five candidate models were tested head-to-head on identical data (linear regression, a regularized linear variant, two different tree-based approaches, and a blend):

Model	Batting R^2	Pitching R^2
Ridge (linear)	**0.375**	0.164
ElasticNet (linear, auto-tuned)	0.375	0.164
Random Forest (tree-based)	0.344	0.219
XGBoost (tree-based)	0.346	**0.219**
Ridge + XGBoost blended	0.370	0.208

Batting: the simplest model (linear regression) won outright. Nothing more complex beat it — a strong sign that batting WAR really is close to a straightforward, additive combination of the underlying stats, which is exactly the kind of relationship linear models are built for.

Pitching: two independently-built tree-based models landed on the identical improvement over linear regression. Two different algorithms agreeing is much stronger evidence than either alone — it means pitching performance has real non-linear structure (some kind of interaction between stats, not just their sum) that a straight line genuinely cannot capture. **Shipped decision: linear regression for batters, a tree-based model (XGBoost) for pitchers** — not an inconsistency, a deliberate response to what the data actually showed for each group.

Stage 4: Valuation — Turning a Prediction Into a Dollar Figure

The formula, in words: for each of the next three seasons, take the predicted WAR, convert it to a dollar figure, discount it slightly for uncertainty the further out it is, then subtract what the player is actually being paid that season. Add the three years together.

> **Surplus Value = the sum, over 3 years, of: (Projected WAR that year × $8,000,000 ÷ 1.08^(years from now)) – Salary that year**

Every constant in that formula is a deliberate, defensible choice, not an arbitrary number.

DESIGN DECISION: $8 MILLION PER WAR

This is the going market rate teams effectively pay for a win, estimated from free-agent contract analysis (this figure is commonly cited in the $8M-$10M range in sabermetric literature covering recent free-agent classes). **Interview-ready answer if pressed on sensitivity:** the exact number moving within that $8M-$10M range doesn't meaningfully change *rankings* — it scales every player's surplus value roughly proportionally, so who looks like a good trade target relative to everyone else barely shifts.

DESIGN DECISION: AN 8% DISCOUNT RATE

Future seasons are worth slightly less than the current one — both because projections get less certain the further out they go, and because money now is worth more than the same money later (ordinary time-value-of-money reasoning). 8% sits in the middle of what's commonly used for this kind of multi-year sports valuation; the system is not especially sensitive to small changes in this number either.

DESIGN DECISION: A 3-YEAR HORIZON, NOT LONGER

Projection accuracy degrades fast the further into the future you go — by year 4 or 5, the uncertainty is large enough that the number stops being trustworthy. Capping at 3 years is an honest acknowledgment of that limit, not an arbitrary stopping point.

DESIGN DECISION: A SIMPLE, EXPLAINABLE AGING CURVE — NOT A LEARNED ONE

A player's performance doesn't stay flat — it typically peaks around age 27 and declines afterward, with the decline accelerating the further past peak a player gets. Rather than training a separate model to learn this shape from data, it's implemented as a straightforward formula: flat through age 27, then a decline rate that increases linearly with distance past peak. This is a **deliberate simplification, stated honestly as a limitation** rather than disguised as something more rigorous than it is — a learned aging curve would need a large, carefully-controlled dataset to build well (and would face its own bias, since only players good enough to still be playing at older ages show up in the data at all — "survivorship bias").

One subtlety worth having ready: the aging curve is *never* applied to year one of the projection. Year one is already a single-season prediction from the forecast model — applying an aging adjustment on top of it would be double-counting the same effect twice. The curve only touches years two and three, carried forward from year one.

DESIGN DECISION: HOW SALARY DATA ACTUALLY GETS SOURCED (A REAL DEBUGGING STORY)

This took a couple of attempts, and it's a genuinely useful thing to walk through if asked "how did you get the salary data":

1. **First attempt: a well-known public salary database.** This turned out to be a dead end — the public source it downloads from no longer exists (confirmed directly: the repository it points to returns a 404). Not a bug in the code, just an unmaintained external dependency that quietly died.
2. **What actually worked:** the same historical data source already being pulled in Stage 1 for actual WAR *also* includes a salary field, for free — including **current-season salary for players already under contract.** Spot-checked against real, publicly known contract figures and confirmed accurate.
3. **The real remaining gap:** that free data only covers the *current* season. A multi-year contract's future guaranteed money isn't in there — the source only reports what a player is actually being paid *right now*. For years two and three of the horizon, that has to be manually looked up (from public contract- tracking sites) for whichever specific players are actually being evaluated — not the whole league, just the ones under consideration.
4. **The fallback, stated honestly:** any player-year still missing real salary data defaults to the MLB league minimum, and every such case is explicitly flagged in the output (`salary_estimated = True`) rather than silently blended in as if it were a real number.

Stage 5: Recommend — Finding Realistic Trades

The goal: given every player's surplus value, suggest realistic 1-for-1 trades between two different teams.

DESIGN DECISION (AND A GENUINE INSIGHT WORTH HAVING READY): WHY "BOTH SIDES WIN" IS MATHEMATICALLY IMPOSSIBLE HERE

The first instinct for a trade finder is "find trades where both teams come out ahead." Working through the math carefully reveals that's actually **structurally impossible** with a single universal value number, except by pure coincidence:

> Surplus value doesn't depend on which team holds the player — it's the same number no matter who owns them. That means a 1-for-1 trade is inherently **zero-sum**: whatever surplus Team A gains by receiving a player worth $60M is exactly what Team B loses by giving that player up. There's no way for a single shared value number to make a genuinely *unequal* trade look good to both sides at once — real-world "win-win" trades only make sense because of team-specific context (a rebuilding team values a cheap prospect more than the raw number suggests; a team about to make the playoffs values immediate production more), which this system deliberately doesn't model.

The honest, correct thing to build instead: find pairs of players on **different teams whose surplus values are closest together** — the nearest thing to a "fair," mutually plausible trade that a value-only model can actually produce. This isn't a downgrade from the original goal, it's a more honest version of it.

DESIGN DECISION: TEAM AFFILIATION FROM THE PROJECTION DATA, NOT A SEPARATE ROSTER TABLE

Rather than building and maintaining a separate up-to-date roster table, each player's current team is read directly from the incoming season projection data, which already includes it. One less thing to keep in sync.

HONEST LIMITATION, STATED UP FRONT

There's currently no check on whether the two players in a "fair" trade even play compatible positions — a relief pitcher and a catcher with matching dollar values will show up as a candidate pair, even though no real front office would consider that swap. This is a known, stated gap (adding positional-need modeling), not a hidden one.

Part 5: How Do We Know Any of This Is Actually Good?

This section exists because of a fair, pointed question raised partway through building this: *"How do you actually evaluate this pipeline, or the model?"* The original answer — "the top of the predicted list has recognizable star names on it" — is **not real evidence**. A model that just rewards "more plate appearances and better raw stats" would produce that same result; stars tend to be good at everything simultaneously. Real evaluation requires comparing specific predictions against specific known-correct answers, many times, with an honest score attached.

The critical distinction: reconstructing the past vs. forecasting the future

Two very different tests were built, and they produce very different-looking numbers:

Test 1 — same-season reconstruction. Give the model a player's real, *already completed* stat line for a season, and ask it to calculate that *same season's* WAR.

Test 2 — true forecast. Give the model only what a player did in a *previous* season, and ask it to predict what they'll do in the *next* one — a season the model has never seen any data from.

	Same-season reconstruction	True forecast (the real job)
Batting R^2	0.82	**0.375**
Pitching R^2	0.56	**0.219**

The first test is deceptively easy: WAR is partly *calculated from* the same stats being fed in, so it's closer to solving a known equation than genuinely predicting anything. The second test is the honest one, because it's the exact situation the real pipeline is actually in — when it runs on next season's incoming projections, it has never seen any real data from that season, exactly like the true-forecast test setup.

Nothing "broke" between these two numbers. The model didn't get worse — the question asked of it got substantially harder and more honest.

BOTH MODELS STILL CLEARLY BEAT DOING NOTHING

	Naive: guess league average	Naive: guess last season repeats	Model
Batting R^2	-0.025	0.220	**0.375**
Pitching R^2	-0.000	-0.210	**0.219**

A model that can't beat these dumb comparison points isn't adding value, no matter how sophisticated it looks. Both models clearly do.

One notable side-finding: for pitchers, “assume they repeat last season” scored *worse* than just guessing the league average. That’s not a flaw in the model — it’s real evidence that pitcher performance genuinely swings harder year to year than hitter performance does, in real life, before any model gets involved (arm injuries, role changes, and batted-ball luck all play a bigger part for pitchers).

What the ablation tests proved

An “ablation” means: change exactly one thing, hold everything else identical, and measure whether accuracy actually improves — instead of guessing.

1. **Does the fielding feature actually help?** Yes — R^2 dropped from 0.375 to 0.331 when it was removed, confirmed under the harder true-forecast test, not just the easier one.
2. **Do “skill-based” pitching stats (strikeout/walk/home-run rates) beat the current ones (ERA/WHIP/wins/losses)?** A wash in the true forecast (R^2=0.164 vs. 0.162) — no change made, since there was no real evidence to justify one. The reason is subtle and worth remembering: the pitching WAR being predicted is itself calculated from actual runs allowed, so ERA/WHIP are almost *definitionally* close to the target, which is a different question from which stats better predict *future* performance.
3. **Does a fancier model beat a simpler one?** Split result, by position group: no gain for batting (simplest model won outright); a real, twice-confirmed gain for pitching (two independent tree-based algorithms agreed).
4. **Are there unused inputs already sitting in the data?** Yes — age and several batting counting stats were either unused or silently dropped during ingestion; adding them produced a real, if modest, accuracy gain, and are now shipped.

A real bug caught along the way

While building the evaluation harness, a data-quality issue surfaced: the training query joined two tables in a way that silently duplicated about 9.4% of training rows whenever a player was traded mid-season (their WAR gets recorded once per team-stint, and the join wasn’t summing across those stints first). Fixed by aggregating WAR per season before joining. **Model accuracy actually improved slightly as a side effect** of the fix, which is a useful thing to say if asked about data quality practices — cleaning up a real bug isn’t neutral, it can directly move the number that matters.

Part 6: Interview Defense — Hard Questions, Prepared Answers

"IS THE MODEL ACTUALLY GOOD?"

"It's good at a specific, honestly-measured job: turning a player's known stat line into an accurate value estimate, and forecasting a season ahead with real, measurable signal that clearly beats naive baselines — R^2=0.375 for hitters, 0.219 for pitchers, when actually forecasting rather than reconstructing a known past season. It's not as accurate as a system with full pitch-level tracking data and a dedicated data team, because I don't have that data. But I built two separate evaluations specifically so I wouldn't fool myself about which number was the real one, and I can tell you exactly why each number is what it is."

"WHY ARE YOUR NUMBERS LOWER THAN PROFESSIONAL SYSTEMS LIKE STEAMER OR ZIPS?"

"Forecasting a season of human athletic performance a year out is genuinely hard for everyone — nobody in this field is near-perfect. My feature set is intentionally minimal: basic box-score stats, not pitch-level Statcast data, multi-year weighted history, or minor-league translations that professional systems use. Given that constraint, the model still clearly beats naive baselines, which is the real bar for 'is this adding value.' More importantly, I deliberately built the harder evaluation and reported that number instead of the easier, inflated one — catching your own model looking better than it really is, on purpose, is a stronger signal of understanding than a high number by itself."

"WHY NOT JUST RANK PLAYERS BY RAW WAR INSTEAD OF BUILDING SURPLUS VALUE?"

"Because the best player and the best trade target are different questions. Ranking by WAR would put a $40M-a-year superstar at the top of every list, but a team can't actually extract a bargain by trading for someone already being paid full market value. Surplus value specifically measures 'production relative to cost' — it's why a cheap, cost-controlled 24-year-old can outrank a superstar as a trade target even though the superstar is the better player."

"HOW DO YOU KNOW YOU'RE NOT ACCIDENTALLY USING FUTURE INFORMATION?"

"Every fact in the database is stamped with when it was actually known (`as_of_date`). This wasn't optional — I specifically caught and fixed a case where a fielding feature would have used a player's same-season defensive performance to predict that same season's value, which is a direct example of the leak this discipline exists to prevent. The evaluation harness enforces the same rule: the true-forecast test only ever trains on seasons strictly before the one being predicted."

"WALK ME THROUGH THE SURPLUS VALUE FORMULA."

"For each of the next three seasons: take the predicted WAR, multiply by $8 million — the going market rate per win — discount it slightly for a season further out using an 8% rate, then subtract what the player is actually paid that season. Sum the three years. Year one comes straight from the WAR model; years two and three

get carried forward using a simple aging curve, never year one itself, since that would double-count the model's own aging adjustment."

"WHY RIDGE FOR BATTERS BUT XGBOOST FOR PITCHERS — ISN'T THAT INCONSISTENT?"

"It's the opposite of inconsistent — it's a deliberate response to what the data actually showed. I tested five different model types on both groups using identical features and an identical held-out test. For batters, the simplest model won outright — batting value really does behave close to a straightforward, additive combination of stats. For pitchers, two independently built tree-based models landed on the exact same improvement over the linear model, which is much stronger evidence than either alone that pitching has real non-linear structure a straight line can't capture. Using the same model everywhere out of a false sense of consistency would have meant leaving real accuracy on the table for pitchers."

"WHAT'S THE BIGGEST WEAKNESS OF THIS SYSTEM?"

"Two, honestly. First, the trade recommender ranks purely by dollar value with no positional-fit check — it can suggest a catcher for a reliever if the numbers happen to line up, which no real front office would do. Second, and more fundamentally, the whole system uses one universal surplus-value number with no team-specific context — no positional need, no contention-window modeling — so it can identify fair trades but structurally cannot identify trades that benefit both sides more than a fair swap would, which is what real-world trades are usually built around."

"WHAT WOULD YOU DO NEXT WITH MORE TIME?"

"Three things, in priority order: add positional-fit filtering to the trade recommender, since that's the fastest fix to the most obvious current gap; build a proper backtest harness that replays a past season using only the data that would have actually been known at that point in time, to validate that a recommendation would have held up in hindsight — separate from the model-accuracy evaluation I already built; and finish the API layer so this is queryable over HTTP instead of only runnable as command-line scripts."

Part 7: Key Numbers — Quick Reference

Constant / Result	Value	Where it's used
Dollars per WAR	$8,000,000	Valuation formula
Discount rate	8%	Valuation formula
Valuation horizon	3 years	Valuation formula
Aging curve peak age	27	Aging curve
League minimum salary fallback	$760,000	Valuation, missing-salary fallback
Entity resolution match rate	793 / 803 (98.8%)	Resolve stage
Training data span	2014–2025 (12 seasons)	Forecast training
Batting R^2, same-season	0.82	Easier, less honest test
Batting R^2, true forecast	**0.375**	The number that matters
Pitching R^2, same-season	0.56	Easier, less honest test
Pitching R^2, true forecast	**0.219**	The number that matters
Batting model	Ridge (linear regression)	Confirmed best of 5 tested
Pitching model	XGBoost (tree-based)	Confirmed best of 5 tested, twice
Fielding feature accuracy gain	R^2 0.331 → 0.375	Confirmed via ablation

Part 8: Honest Limitations (Full List)

- The aging curve is an explainable heuristic, not learned from real data.
- Multi-player trades (2-for-1 and larger) are entirely out of scope — the search space grows combinatorially, and this project focuses on cleanly demonstrating 1-for-1 valuation instead.
- The trade recommender has no positional-fit or roster-construction awareness.
- There's no team-specific value modeling (contention window, positional need), which is the reason "fair trades" is the right framing instead of "win-win trades."
- No authentication, frontend, or cloud deployment — this is a local, containerized demonstration (Docker + a database + command-line pipeline scripts).
- `data/manual_contracts.csv` (future-year salary for specific players) starts empty and needs to be populated by hand for whichever players are actually evaluated in a trade scenario.
- A formal point-in-time backtest of full *recommendations* (not just model accuracy) — replaying a past season using only data that would have actually been available then — hasn't been built yet.